03 · Context: The Standard "Generate-Then-Encode" Pipeline

The short version

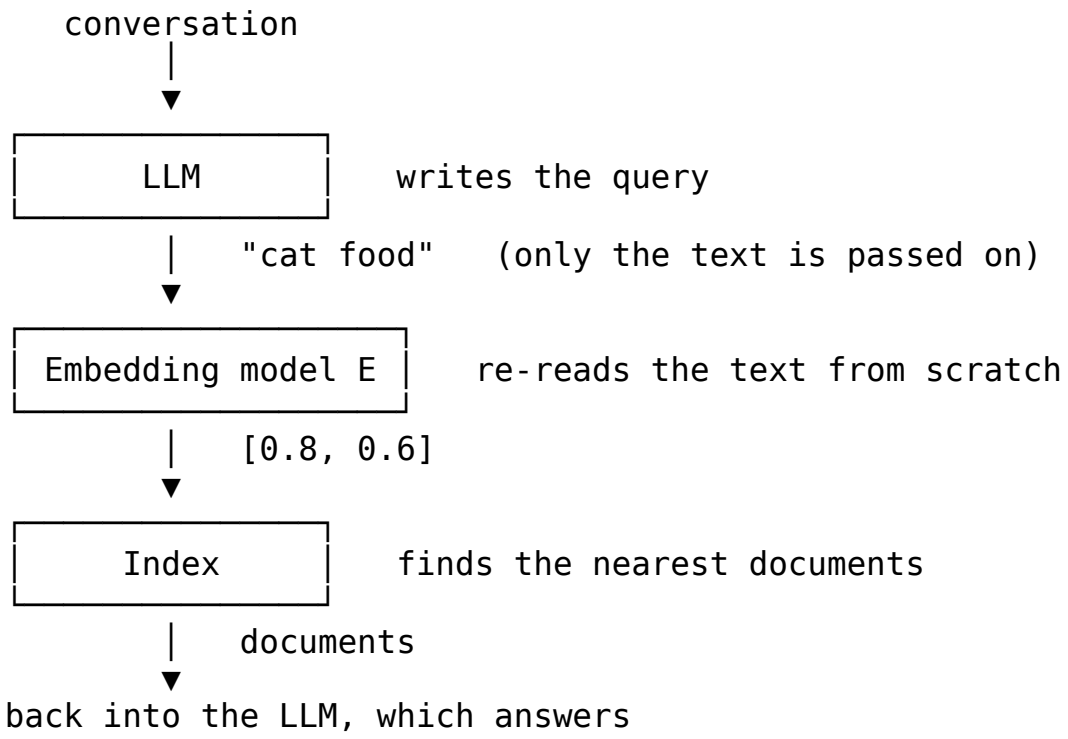
Today most agents search in two stages. The chat model **writes** a search query as text. A separate embedding model **reads** that text and turns it into a vector. The vector is then compared with the index.

The pipeline, step by step

Step	Who does it	What happens	In the toy example
1. Decide to search	LLM	The agent notices it needs outside information.	User asks something about feeding a cat.
2. Write the query	LLM	The agent generates a short, self-contained search query, token by token. This is often called *query rewriting*.	Writes "cat food".
3. Hand over text	—	Only the text string crosses to the next stage. Everything else the LLM knew is left behind.	The string `"cat food"`.
4. Encode	Embedding model E	A *second* model reads the text from scratch and outputs a vector.	$E(\text{"cat food"}) = [0.8, 0.6]$
5. Search	Vector database	Compare the vector with every document vector, return the closest.	"Healthy dry food for cats" wins.

Step	Who does it	What happens	In the toy example
6. Answer	LLM	The retrieved passages are pasted into the LLM's context and it answers.	—

A picture



Where this design came from

It was inherited from search engines and early RAG chatbots, where a query arrived as text from a human and *had* to be encoded by something. Agents copied the pattern even though, in an agent, the query is written by a model that has already "understood" the situation.

In the paper's experiments

This pipeline is the **baseline**:

- LLM: **Qwen3-8B**, writes the query with greedy decoding, up to 32 tokens.
- Embedder: **Qwen3-Embedding-8B**, encodes the query into a 1,024-number vector.
- Search: cosine similarity against a pre-built document index.

The baseline scores Recall@10 = 0.637 and takes about 43.5 ms per query for the encode step (numbers explained in 17 and 18).

Keep this table in mind

Step 3 is the weak point. The next five files are about what goes wrong there and around it.